Lab sheet 06 **Submission Due: 01.07.2024**

Title: Introduction to AJAX.

Aims:

- Grasping Asynchronous Concepts and AJAX Significance
- Mastering Asynchronous Request Handling with XMLHttpRequest
- Proficiency in Asynchronous Request Processing Using the fetch API
- Update content, validate forms, AJAX
- Practical Application and User Experience Enhancement

Tasks:

- Making Asynchronous Requests with XMLHttpRequest.
- Making Asynchronous Requests with the fetch API.
- Updating Web Content Without Page Reload.

Activities :

1. Making Asynchronous Requests with XMLHttpRequest
   I. Example 01 :

```
<script>
function loadDoc() {
  var xhttp = new XMLHttpRequest();
  xhttp.onreadystatechange = function() {
    if (this.readyState == 4 && this.status == 200) {
      document.getElementById("demo").innerHTML =
      this.responseText;
    }
  };
  xhttp.open("GET", "info.txt", true);
  xhttp.send();
}
</script>
```

   II. Example 02 :

```
document.getElementById("fetchContent").addEventListener("click",
fetchContentWithXMLHttpRequest);
function fetchContentWithXMLHttpRequest() {
 var xhr = new XMLHttpRequest();
 xhr.open("GET", "https://www.w3schools.com/html/html_intro.asp",
true);
 xhr.onreadystatechange = function () {
 if (xhr.readyState === 4) {
 if (xhr.status === 200) {
```

```
 displayContent("output", xhr.responseText);
 } else {
 displayError("output", "Failed to fetch content with
XMLHttpRequest.");
 }
 }
 };
 xhr.send();
}
function displayContent(targetId, content) {
 var outputElement = document.getElementById(targetId);
 outputElement.textContent = content;
}
function displayError(targetId, errorMessage) {
 var outputElement = document.getElementById(targetId);
 outputElement.textContent = "Error: " + errorMessage;
}
```

2. Making Asynchronous Request with the fetch API.

```
document.getElementById("fetchContent").addEventListener("click",
fetchContentWithFetchAPI);
function fetchContentWithFetchAPI() {
 fetch("https://www.w3schools.com/html/html_intro.asp")
 .then(response => {
 if (response.ok) {
 return response.text();
 } else {
 throw new Error("Failed to fetch content with fetch API.");
 }
 })
 .then(content => {
 displayContent("output", content);
 })
 .catch(error => {
 displayError("output", error.message);
 });
}
function displayContent(targetId, content) {
 var outputElement = document.getElementById(targetId);
 outputElement.textContent = content;
}
function displayError(targetId, errorMessage) {
 var outputElement = document.getElementById(targetId);
 outputElement.textContent = "Error: " + errorMessage;
}
```

3. Updating Web Content Without Page Reload

```
document.getElementById("submitForm").addEventListener("click",
submitForm);
function submitForm() {
 var form = document.getElementById("myForm");
 var name = form.elements["name"].value;
 var email = form.elements["email"].value;
 var outputElement = document.getElementById("formOutput");
 if (name && email) {
 var formData = new FormData(form);
 var xhr = new XMLHttpRequest();
 xhr.open("POST", "process.php", true);
 xhr.onload = function() {
 if (xhr.status === 200) {
 outputElement.innerHTML = xhr.responseText;
 } else {
 outputElement.innerHTML = "Form submission failed.";
 }
 };
 xhr.send(formData);
 } else {
 outputElement.innerHTML = "Please fill in all fields.";
 }
}

setTimeout(function () {
 var contentElement = document.getElementById("content");
 contentElement.textContent = "This is updated content using AJAX!";
}, 5000);
```

Discussion:

- readyState property
- onreadystatechange property
- HTTP status messages